Universidad ORT Uruguay
Facultad de Ingeniería

Entregado como requisito para la obtención del título de
Ingeniería en Sistemas

Juan Pedro Ferreira (292400)
Matías Trujillo (272561)
Agustín Lanata (285326)

Entrega 2
Diseño de Aplicaciones 2

2025

Índice

Descripción y decisiones del diseño **3**
Descripción de la solución 3
Reflection 3
Errores y Bugs a solucionar: 4
Vista Lógica **4**
Dominio 5
Data Access 7
Descripción del mecanismo de acceso a datos utilizado. 8
Business Logic 8
Web Api 14
Diagrama DTO 16
Diagramas de Secuencia 16
Vista de Desarrollo **20**
Diagrama de paquetes 20
DTOs 22
WebApi 22
Métricas: 22
Diagrama de componentes 27
Modelo de Tablas 28
Mejoras a nivel de diseño 28
Anexo: **29**

Descripción y decisiones del diseño

Descripción de la solución

La solución desarrollada consiste en un Simulador de Objetos que permite modelar y ejecutar estructuras propias de la programación orientada a objetos (OOP) de forma dinámica. A través de una API RESTful.

Por otro lado, el simulador permite definir clases que pueden ser abstractas o selladas, e indicar si se heredan de otras clases o interfaces, se asume herencia implícita de la clase Object, para esto decidimos generar esta clase como Seed Data desde la clase SqlContext que se encuentra en el paquete de DataAccess, al igual que hicimos con los tipos primitivos, incluyendo sus métodos, como en el entorno .NET. Cada clase puede tener atributos y métodos. En el caso de los métodos, además se especifica el tipo de retorno, si son abstractos o sellados, y se pueden asociar con parámetros, variables locales y una lista de métodos invocados, también al comenzar la funcionalidad se consideró necesaria la clase “InvokedMethod” que dado el id de un método se especifica cómo será su invocación, sea desde un atributo, variable local, parametro, this o base.

Una parte fundamental del sistema es la simulación de ejecución de métodos. Para ello, el usuario define un tipo de referencia y un tipo concreto de objeto instanciable. Luego, selecciona un método a ejecutar, y el sistema muestra la traza de ejecución paso a paso, reflejando las invocaciones internas que realiza el método, ya sean a través de this, base, atributos, parámetros o variables locales. Durante esta ejecución, se valida que las invocaciones respeten las reglas de visibilidad y el alcance definido en el modelo. Además, decidimos usar recursión para recorrer los métodos y sus invocaciones.

El sistema cuenta con una arquitectura robusta, incluyendo un filtro global de excepciones que devuelve respuestas HTTP apropiadas (400, 404, 409, 500) según el tipo de error detectado. Esto mejora la usabilidad y confiabilidad del simulador, además de facilitar el desarrollo y el mantenimiento. La implementación modular y extensible permite que se puedan agregar nuevas funcionalidades en el futuro, como soporte para miembros estáticos o nuevos modificadores de acceso.

Reflection

En la clase ExporterService se implementó una funcionalidad basada en reflection que permite descubrir e instanciar dinámicamente exportadores definidos en ensamblados externos. A través del uso de reflexión, se evita la dependencia directa de clases concretas. Sin embargo, debido a cómo se encuentra implementado actualmente el método

ExecuteMethod, la clase TransformerForExporter depende del dominio, lo que significa que cualquier desarrollador que quiera agregar una nueva DLL exportadora necesita conocer detalles del dominio. Una forma más limpia de evitar este acoplamiento sería separar la parte del dominio necesaria (la clase Attribute) en una DLL independiente que pueda ser consumida sin necesidad de referenciar todo el dominio.

Para lograr esta funcionalidad dinámica, se definió una interfaz común llamada ITransformerExporter, la cual debe ser implementada por cualquier clase que actúe como exportador. Estas implementaciones se compilan en archivos .dll que se colocan dentro de una carpeta específica llamada Exporters, dentro del proyecto Simulator.WebApi. Un ejemplo concreto de esto es el ensamblado Simulator.HTMLExporter, que podría estar incluso fuera de la solución principal. Dentro de este se encuentra la clase Class1. Esta clase implementa el método Export, que toma una cadena de texto y la transforma en un documento HTML. Además, implementa GetFormat, que retorna "html", permitiendo al sistema identificar el tipo de exportador.

El método LoadAssemblies de ExporterService recorre la jerarquía de carpetas hasta encontrar la raíz del proyecto, luego accede a la carpeta Exporters y carga todos los ensamblados .dll que encuentra. A continuación, los métodos ExportTransfer y GetAvailableExporters inspeccionan estos ensamblados para encontrar todas las clases que implementan la interfaz ITransformerExporter. En el caso de ExportTransfer, se selecciona dinámicamente la clase que coincide con el tipo de exportación solicitado (por ejemplo, "html") y se instancia con Activator.CreateInstance. Luego, se ejecuta su método Export pasándole como argumento los datos generados por el _methodService. El método GetAvailableExporters, por su parte, permite listar todos los formatos disponibles preguntando a cada exportador su resultado de GetFormat.

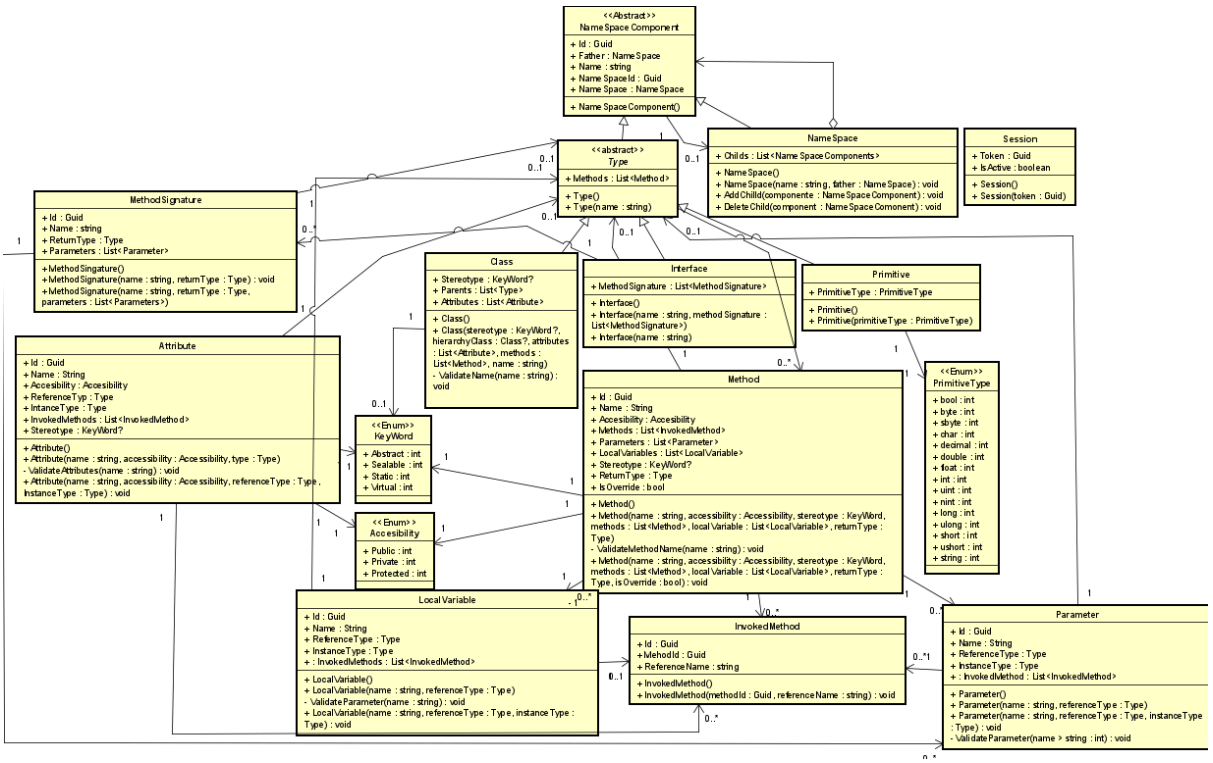
Errores y Bugs a solucionar:

1) El orden de ejecución en ExecuteMethod no depende del orden de entrada, sino por algún orden que tiene la base de datos.

Vista Lógica

Está enfocada en el diseño funcional del sistema, detallando cómo se descomponen los requisitos en clases y objetos, y cómo se relacionan entre sí. Está orientada al arquitecto y sirve como base para la implementación.

Dominio



El paquete Domain se encarga de definir el modelo de negocio central del sistema. Contiene las entidades principales, sus propiedades y comportamientos.. Representa lo que el sistema "conoce" y "manipula", sin incluir detalles de lógica de aplicación o infraestructura.

Para representar tanto los tipos primitivos como los tipos por referencia de forma estructurada, se definió una clase base llamada Type. Esta clase contiene propiedades comunes a ambos, como el nombre del tipo. Luego, se tienen las clases Interface, Primitive y Class que hereda de Type y agregan su propio comportamiento específico según corresponda.

Gracias a esta herencia, es posible aplicar polimorfismo, lo que significa que se puede trabajar con objetos de tipo Primitive, interface o Class utilizando una referencia del tipo base Type. Esto permite escribir código más flexible y reutilizable ,ya que no es necesario conocer el tipo exacto del objeto para invocar los métodos comunes definidos en la clase base. Por ejemplo, una lista de tipos (List<Type>) puede contener instancias de ambas clases (Primitive y Class) y tratarlas de forma uniforme.

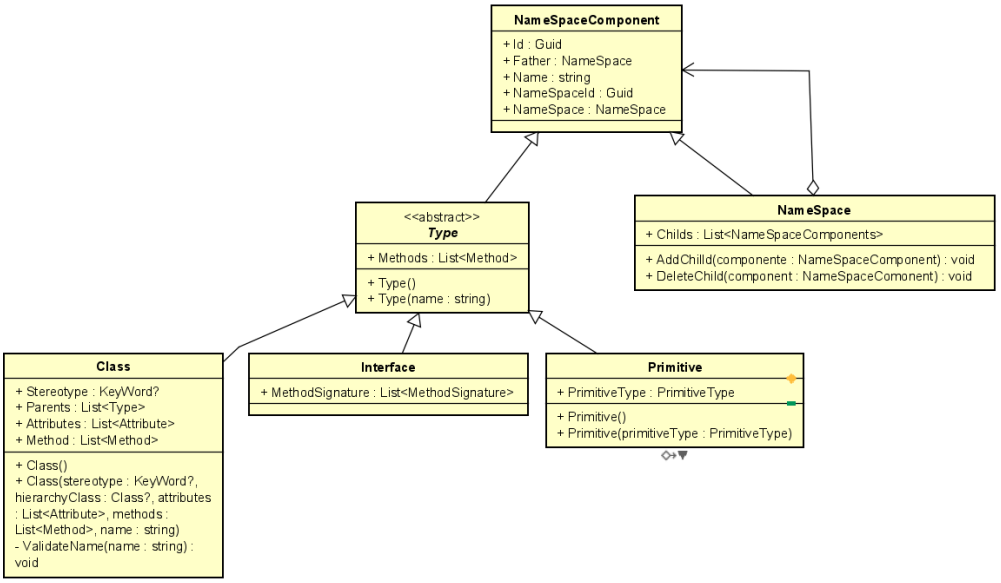
Esta estrategia simplifica el manejo de los diferentes tipos dentro del sistema, permite por ejemplo que el tipo de un atributo sea tanto una clase como un tipo primitivo.

Aunque actualmente se hayan definido todos los valores posibles, como los tipos primitivos disponibles en el lenguaje, el uso de enum sigue siendo beneficioso, ya que centraliza las opciones válidas, mejora la legibilidad del código y garantiza la seguridad de tipo en tiempo de compilación. En caso de que en el futuro se necesiten nuevos tipos, niveles de acceso o

palabras clave, basta con agregar valores al enum correspondiente, sin modificar estructuras complejas ni lógica dispersa.

Utilización del Patrón Composite en la funcionalidad “NameSpace”

En este diseño se aplica el **patrón de diseño estructural Composite** para representar estructuras jerárquicas de componentes que forman parte de un espacio de nombres.

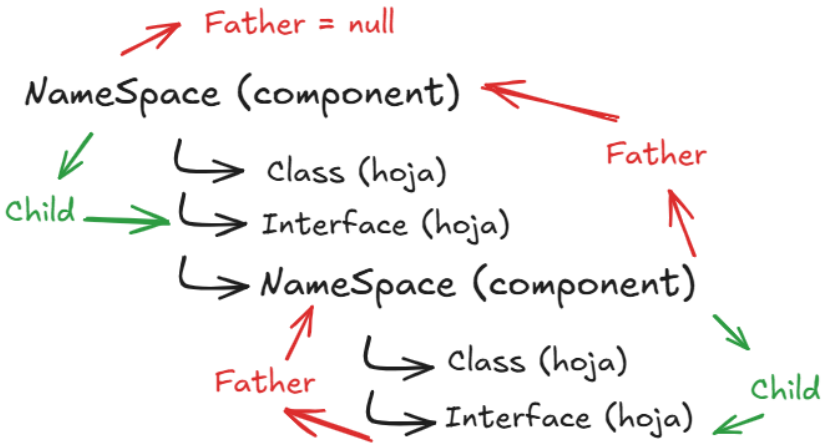


- La clase abstracta NameSpaceComponent actúa como la superclase común para todos los elementos que pueden formar parte de un NameSpace.
- La clase NameSpace contiene una colección de NameSpaceComponent, lo que le permite contener otros namespace o componentes, estableciendo así una estructura de árbol.
- NameSpace implementa métodos como AddChild y DeleteChild para gestionar esta jerarquía.

Uniformidad en el tratamiento: El patrón permite que tanto los elementos individuales como las composiciones (otros espacios de nombres que contienen componentes) sean tratados de forma uniforme. Por ejemplo, se puede recorrer un NameSpace y operar sobre sus NameSpaceComponent sin saber si se trata de una clase, interfaz, primitivo u otro espacio de nombres.

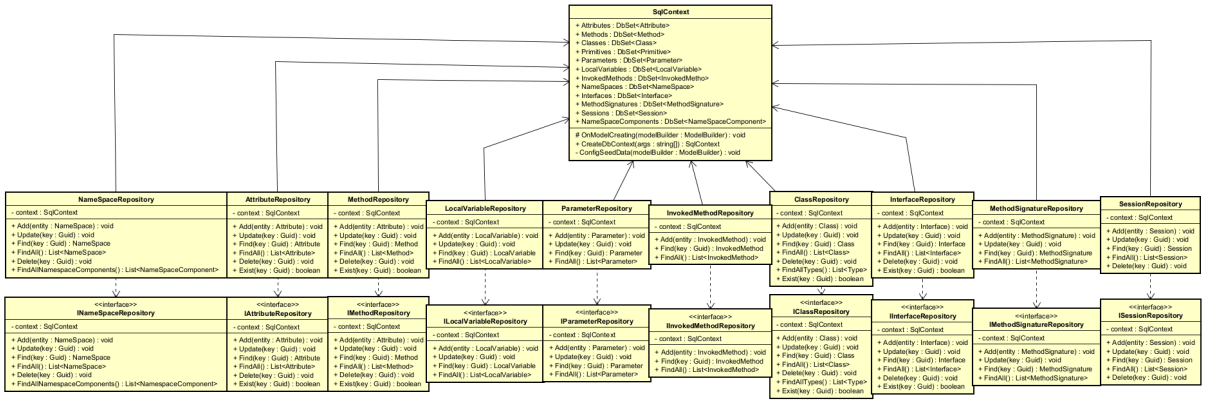
Facilidad de extensibilidad: Si se desea agregar un nuevo tipo de componente (por ejemplo, una enumeración o estructura), solo se necesita crear una nueva clase que herede de NameSpaceComponent y se integrará fácilmente en la jerarquía.

Ejemplo visual del manejo de jerarquías



Data Access

En este diagrama se ven representados tanto el paquete DataAccess que consta de todas las clases concretas y el paquete IDataAccess que son todas las interfaces que se implementan.



El paquete DataAccess es responsable de la gestión de la persistencia de datos en el sistema. Proporciona las interfaces y clases necesarias para interactuar con la base de datos, incluyendo las operaciones CRUD . Este paquete implementa la capa de acceso a datos de la aplicación, manteniendo la separación de responsabilidades entre la lógica de negocio y la persistencia de datos.

Repositorios: Contiene las clases que implementan los repositorios de las entidades del dominio, proporcionando métodos para acceder y manipular los datos en la base de datos. Los repositorios son responsables de interactuar con el contexto de la base de datos usando Entity Framework y ejecutar consultas para agregar,recuperar, actualizar o borrar los datos.

SqlContext: Define el contexto de la base de datos, que incluye las conexiones, entidades y configuraciones necesarias para interactuar con la base de datos. Esta clase hereda de DbContext y mapea las entidades a las tablas correspondientes en la base de datos.

Migraciones: Este paquete también gestiona las migraciones de la base de datos, permitiendo que los cambios en el esquema de la base de datos se reflejen de manera controlada y versionada.

Descripción del mecanismo de acceso a datos utilizado.

El mecanismo de acceso a datos utilizado en el proyecto se basa en Entity Framework Core (EF Core) como ORM (Object-Relational Mapper), específicamente con el proveedor de SQL Server. Para gestionar la persistencia de los objetos del dominio, se utiliza una clase DbContext personalizada denominada SqlContext. Esta clase define un conjunto de DbSet<TEntity> que representan las entidades principales del modelo de dominio, como Class, Method, Attribute, entre otras.

Dentro del método OnModelCreating, se configuran las relaciones entre las entidades, las claves primarias, y las relaciones de herencia mediante el uso de Fluent API. También se incluye la carga de datos iniciales (seed data).

Se ha habilitado el uso de Lazy Loading Proxies a través de la configuración en OnConfiguring, lo cual permite cargar automáticamente entidades relacionadas al acceder a sus propiedades de navegación.

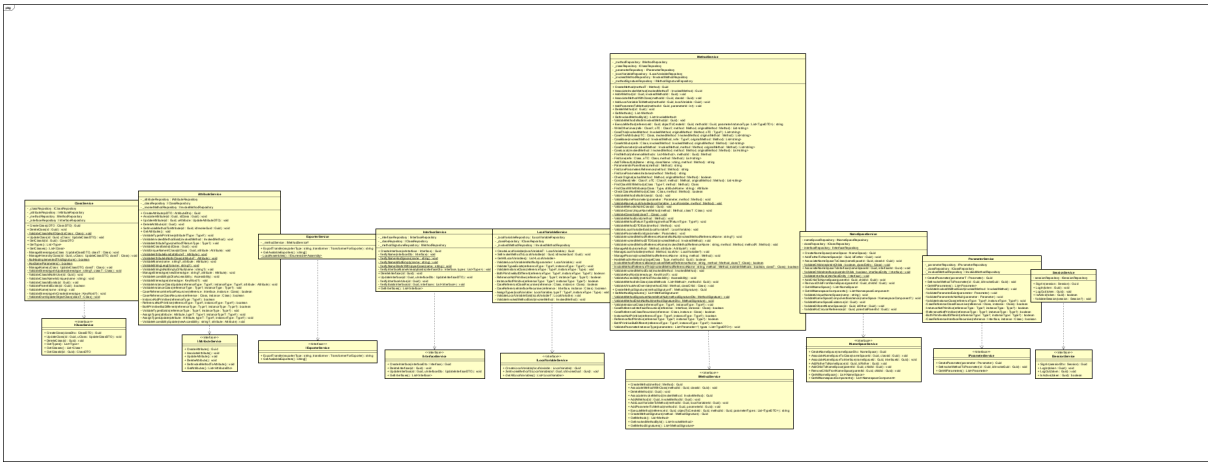
Si bien en nuestra implementación actual la cadena de conexión está escrita de forma explícita dentro de la clase SqlContextFactory, reconocemos que una mejor práctica sería externalizar dicha configuración. Idealmente, se debería obtener la cadena de conexión desde un archivo de configuración como appsettings.json o desde variables de entorno.Esto permitiría mayor flexibilidad al cambiar entre entornos (desarrollo, testing, producción), mejoraría la seguridad al evitar exponer credenciales sensibles en el código fuente, y fomentaría la mantenibilidad al seguir el principio de separación de configuración y lógica.

La aplicación se ejecuta dentro de un entorno Dockerizado. El archivo Dockerfile define una imagen basada en .NET 8 para compilar y ejecutar el proyecto, mientras que el archivo docker-compose.yml orquesta tanto el contenedor de la aplicación como el contenedor de SQL Server. La base de datos se ejecuta en un contenedor separado.

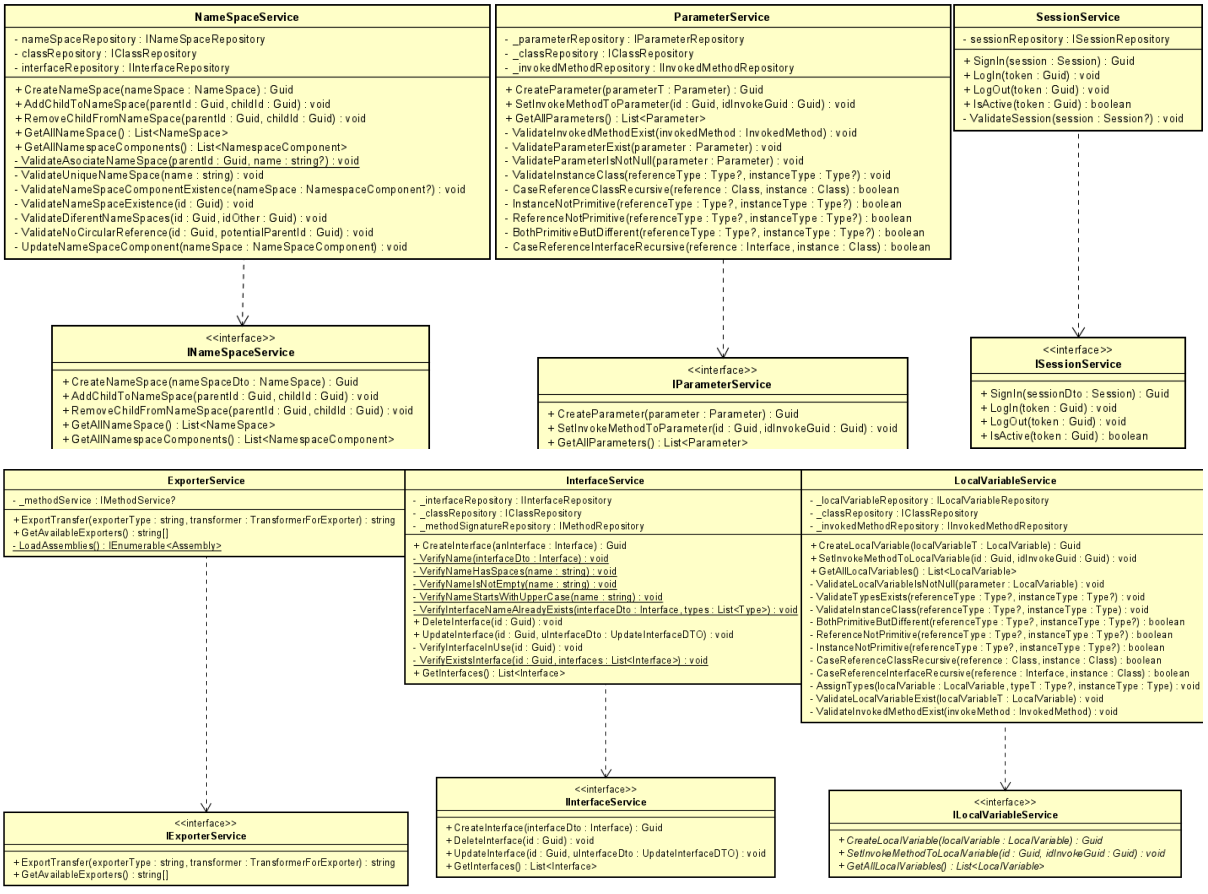
Para generar las migraciones correctamente, se debe posicionar la consola en el proyecto Simulator.DataAcces, ya que allí se encuentra la implementación del contexto SqlContext y la clase SqlContextFactory.

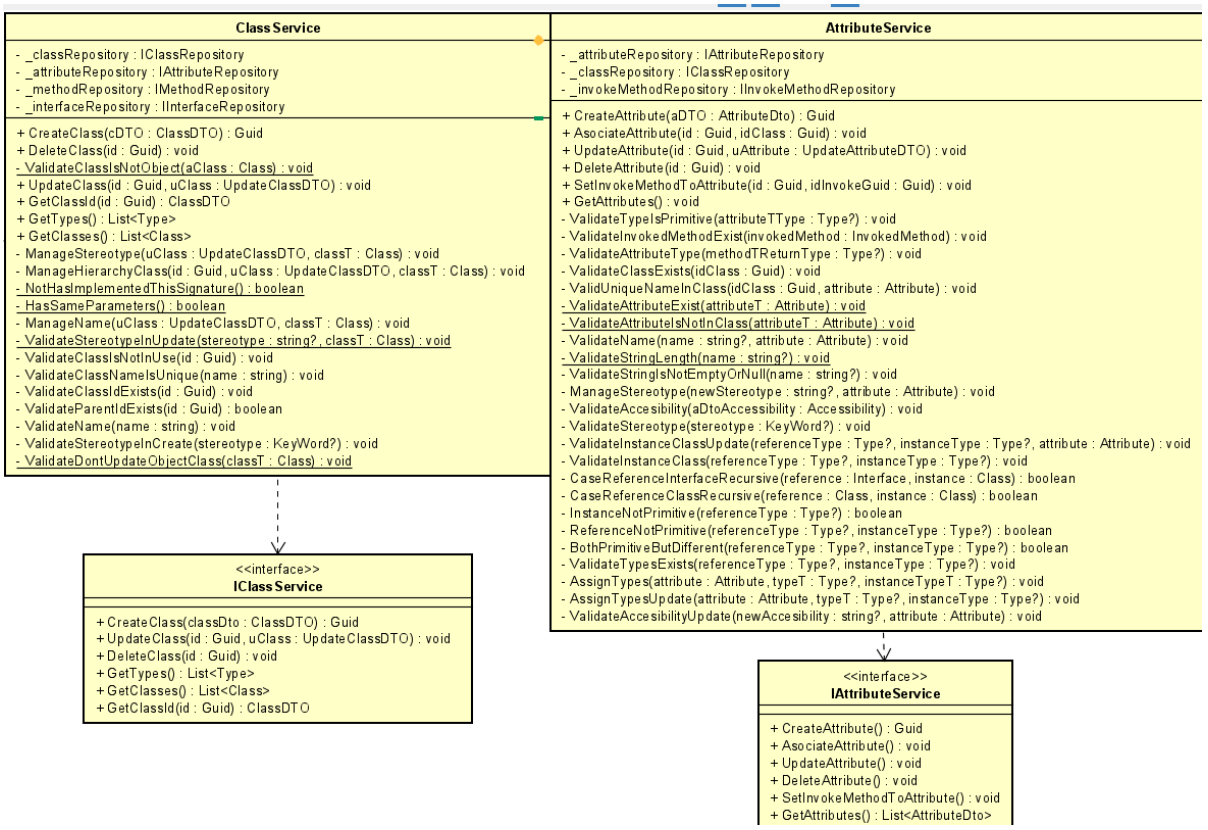
Business Logic

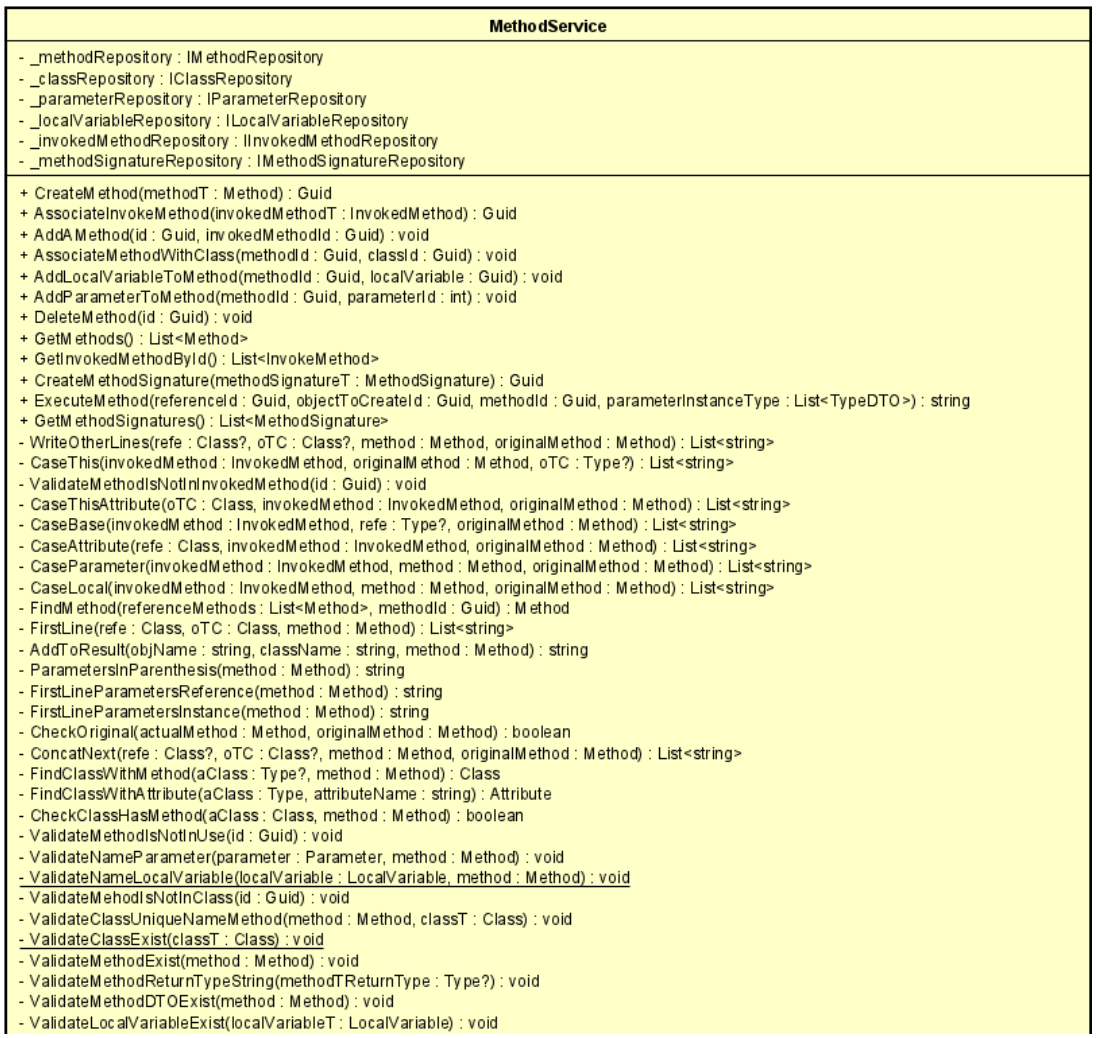
Cabe recalcar que decidimos mostrar este paquete junto al de IBusinessLogic para una mejor claridad.



Fotos con zoom por partes:




ClassService
- _classRepository : IClassRepository
- _attributeRepository : IAttributeRepository
- _methodRepository : IMethodRepository
- _interfaceRepository : IInterfaceRepository
+ CreateClass(cDTO : ClassDTO) : Guid
+ DeleteClass(id : Guid) : void
- ValidateClassIsNotObject(aClass : Class) : void
+ UpdateClass(id : Guid, uClass : UpdateClassDTO) : void
+ GetClassId(id : Guid) : ClassDTO
+ GetTypes() : List<Type>
+ GetClasses() : List<Class>
- ManageStereotype(uClass : UpdateClassDTO, classT : Class) : void
- ManageHierarchyClass(id : Guid, uClass : UpdateClassDTO, classT : Class) : void
- NotHasImplementedThisSignature() : boolean
- HasSameParameters() : boolean
- ManageName(uClass : UpdateClassDTO, classT : Class) : void
- ValidateStereotypeInUpdate(stereotype : string?, classT : Class) : void
- ValidateClassIsNotInUse(id : Guid) : void
- ValidateClassNameIsUnique(name : string) : void
- ValidateClassIdExists(id : Guid) : void
- ValidateParentIdExists(id : Guid) : boolean
- ValidateName(name : string) : void
- ValidateStereotypeInCreate(stereotype : KeyWord?) : void
- ValidateDontUpdateObjectClass(classT : Class) : void
AttributeService
- _attributeRepository : IAttributeRepository
- _classRepository : IClassRepository
- _invokeMethodRepository : IInvokeMethodRepository
+ CreateAttribute(aDTO : AttributeDto) : Guid
+ AsociateAttribute(id : Guid, idClass : Guid) : void
+ UpdateAttribute(id : Guid, uAttribute : UpdateAttributeDTO) : void
+ DeleteAttribute(id : Guid) : void
+ SetInvokeMethodToAttribute(id : Guid, idInvokeGuid : Guid) : void
+ GetAttributes() : void
- ValidateTypeIsPrimitive(attributeTType : Type?) : void
- ValidateInvokedMethodExist(invokedMethod : InvokedMethod) : void
- ValidateAttributeType(methodTReturnType : Type?) : void
- ValidateClassExists(idClass : Guid) : void
- ValidUniqueNameInClass(idClass : Guid, attribute : Attribute) : void
- ValidateAttributeExist(attributeT : Attribute) : void
- ValidateAttributeIsNotInClass(attributeT : Attribute) : void
- ValidateName(name : string?, attribute : Attribute) : void
- ValidateStringLength(name : string?) : void
- ValidateStringIsNotEmptyOrNull(name : string?) : void
- ManageStereotype(newStereotype : string?, attribute : Attribute) : void
- ValidateAccesibility(aDtoAccessibility : Accessibility) : void
- ValidateStereotype(stereotype : KeyWord?) : void
- ValidateInstanceClassUpdate(referenceType : Type?, instanceType : Type?, attribute : Attribute) : void
- ValidateInstanceClass(referenceType : Type?, instanceType : Type?) : void
- CaseReferenceInterfaceRecursive(reference : Interface, instance : Class) : boolean
- CaseReferenceClassRecursive(reference : Class, instance : Class) : boolean
- InstanceNotPrimitive(referenceType : Type?) : boolean
- ReferenceNotPrimitive(referenceType : Type?, instanceType : Type?) : boolean
- BothPrimitiveButDifferent(referenceType : Type?, instanceType : Type?) : boolean
- ValidateTypesExists(referenceType : Type?, instanceType : Type?) : void
- AssignTypes(attribute : Attribute, typeT : Type?, instanceTypeT : Type?) : void
- AssignTypesUpdate(attribute : Attribute, typeT : Type?, instanceType : Type?) : void
- ValidateAccesibilityUpdate(newAccesibility : string?, attribute : Attribute) : void
<<interface>>
IClassService
+ CreateClass(classDto : ClassDTO) : Guid
+ UpdateClass(id : Guid, uClass : UpdateClassDTO) : void
+ DeleteClass(id : Guid) : void
+ GetTypes() : List<Type>
+ GetClasses() : List<Class>
+ GetClassId(id : Guid) : ClassDTO
<<interface>>
IAttributeService
+ CreateAttribute() : Guid
+ AsociateAttribute() : void
+ UpdateAttribute() : void
+ DeleteAttribute() : void
+ SetInvokeMethodToAttribute() : void
+ GetAttributes() : List<AttributeDto>


MethodService
- _methodRepository : IMethodRepository
- _classRepository : IClassRepository
- _parameterRepository : IParameterRepository
- _localVariableRepository : ILocalVariableRepository
- _invokedMethodRepository : IInvokedMethodRepository
- _methodSignatureRepository : IMethodSignatureRepository
+ CreateMethod(methodT : Method) : Guid
+ AssociateInvokeMethod(invokedMethodT : InvokedMethod) : Guid
+ AddAMethod(id : Guid, invokedMethodId : Guid) : void
+ AssociateMethodWithClass(methodId : Guid, classId : Guid) : void
+ AddLocalVariableToMethod(methodId : Guid, localVariable : Guid) : void
+ AddParameterToMethod(methodId : Guid, parameterId : int) : void
+ DeleteMethod(id : Guid) : void
+ GetMethods() : List<Method>
+ GetInvokedMethodById() : List<InvokeMethod>
+ CreateMethodSignature(methodSignatureT : MethodSignature) : Guid
+ ExecuteMethod(referenceId : Guid, objectToCreateId : Guid, methodId : Guid, parameterInstanceType : List<TypeDTO>) : string
+ GetMethodSignatures() : List<MethodSignature>
- WriteOtherLines(refe : Class?, oTC : Class?, method : Method, originalMethod : Method) : List<string>
- CaseThis(invokedMethod : InvokedMethod, originalMethod : Method, oTC : Type?) : List<string>
- ValidateMethodIsNotInInvokedMethod(id : Guid) : void
- CaseThisAttribute(oTC : Class, invokedMethod : InvokedMethod, originalMethod : Method) : List<string>
- CaseBase(invokedMethod : InvokedMethod, refe : Type?, originalMethod : Method) : List<string>
- CaseAttribute(refe : Class, invokedMethod : InvokedMethod, originalMethod : Method) : List<string>
- CaseParameter(invokedMethod : InvokedMethod, method : Method, originalMethod : Method) : List<string>
- CaseLocal(invokedMethod : InvokedMethod, method : Method, originalMethod : Method) : List<string>
- FindMethod(referenceMethods : List<Method>, methodId : Guid) : Method
- FirstLine(refe : Class, oTC : Class, method : Method) : List<string>
- AddToResult(objName : string, className : string, method : Method) : string
- ParametersInParenthesis(method : Method) : string
- FirstLineParametersReference(method : Method) : string
- FirstLineParametersInstance(method : Method) : string
- CheckOriginal(actualMethod : Method, originalMethod : Method) : boolean
- ConcatNext(refe : Class?, oTC : Class?, method : Method, originalMethod : Method) : List<string>
- FindClassWithMethod(aClass : Type?, method : Method) : Class
- FindClassWithAttribute(aClass : Type, attributeName : string) : Attribute
- CheckClassHasMethod(aClass : Class, method : Method) : boolean
- ValidateMethodIsNotInUse(id : Guid) : void
- ValidateNameParameter(parameter : Parameter, method : Method) : void
- ValidateNameLocalVariable(localVariable : LocalVariable, method : Method) : void
- ValidateMehodIsNotInClass(id : Guid) : void
- ValidateClassUniqueNameMethod(method : Method, classT : Class) : void
- ValidateClassExist(classT : Class) : void
- ValidateMethodExist(method : Method) : void
- ValidateMethodReturnTypeString(methodTReturnType : Type?) : void
- ValidateMethodDTOExist(method : Method) : void
- ValidateLocalVariableExist(localVariableT : LocalVariable) : void



La capa de Business Logic se encarga de aplicar las reglas del negocio que rigen el funcionamiento del sistema, coordinando la interacción entre entidades, validando datos y asegurando que las operaciones cumplan con los requisitos funcionales.

Para mejorar la cohesión y seguir el principio de Responsabilidad Única (SRP), se decidió dividir las diferentes funcionalidades del sistema en servicios especializados. Cada servicio se encarga de una responsabilidad específica, a lo largo del proyecto se noto que el service del method se encargaba de más cosas de las que eran necesarias aunque se le quitaron funcionalidades al realizar el executemethod en este servicio quedó más complejo de lo que se esperaba, una posible solucion podria haber sido hacer un servidor que se encargue de toda esa funcionalidad.

El uso de interfaces para cada servicio cumple además con los principios de Segregación de Interfaces (ISP) e Inversión de Dependencias (DIP).

Siguiendo el principio de Segregación de Interfaces (ISP), las interfaces se diseñan para incluir únicamente los métodos que son relevantes para el contexto en el que se utilizan. De esta manera, se evita que los controladores dependan de métodos que no necesitan. Esta práctica reduce la complejidad de las dependencias y mejora la claridad en la comunicación entre los diferentes componentes del sistema. De todos modos MethodService no cumple con este principio ya que tiene bastantes funcionalidades como ya fue mencionado.

Además, al seguir el principio de Inversión de Dependencias (DIP), se logró que los controladores dependieran de abstracciones (interfaces) y no de implementaciones concretas.

Estos servicios no conocen cómo vienen los datos desde el front (únicamente en los casos del update de ciertas entidades) esto favorece el encapsulamiento y el desacoplamiento lo que mejora la mantenibilidad.

Responsabilidad de cada servicio:

Simulator.Attribute Service: Administra los atributos definidos por el usuario dentro del sistema. Su responsabilidad es crear, actualizar, asociar a una clase y eliminar atributos, asegurando que se cumplan las validaciones necesarias en cada operación.

Simulator.Class Service: Se encarga de crear, actualizar y eliminar clases definidas por el usuario. Valida que los nombres sean correctos, que no se repitan y que las clases no estén en uso antes de ser eliminadas o modificadas.

Simulation.Method Service: gestiona toda la lógica relacionada con los métodos dentro del sistema. Se encarga de su creación y asociación de sus parámetros y variables locales. También permite crear invocaciones de métodos, representando cómo estos son llamados dentro del sistema y asociarlos a un método en específico. Además, ofrece funcionalidades para eliminar un método, siempre que este no se encuentre en uso y de la lógica para el ExecuteMethod.
Este servicio se encarga de muchos funcionamiento esto viola el Principio de Responsabilidad Única (SRP) de SOLID, ya que concentra demasiadas tareas distintas en una sola clase, dificultando su mantenimiento y evolución. Una clara mejora es

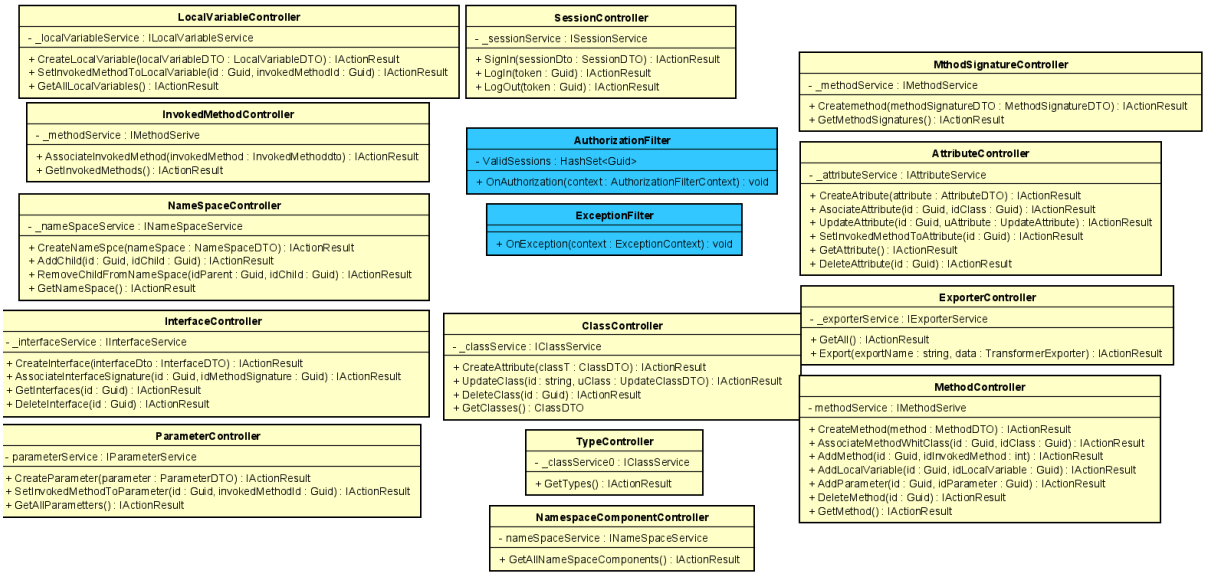
Simulator.Local VariableService y Simulator.Parameter.Service: Gestiona la lógica de estas dos para crear estas entidades y poder setear las invocaciones de métodos.

Simulator.InterfaceService: Permite crear, eliminar, actualizar y obtener interfaces, validando reglas como unicidad de nombre, formato del nombre y que no estén en uso por clases antes de eliminarlas o modificarlas.

Simulator.SessionService: Puede crear una nueva sesión, activar (log in), desactivar (log out) y verificar si una sesión está activa, validando que no existan duplicados y que las operaciones sean coherentes con el estado actual de la sesión.

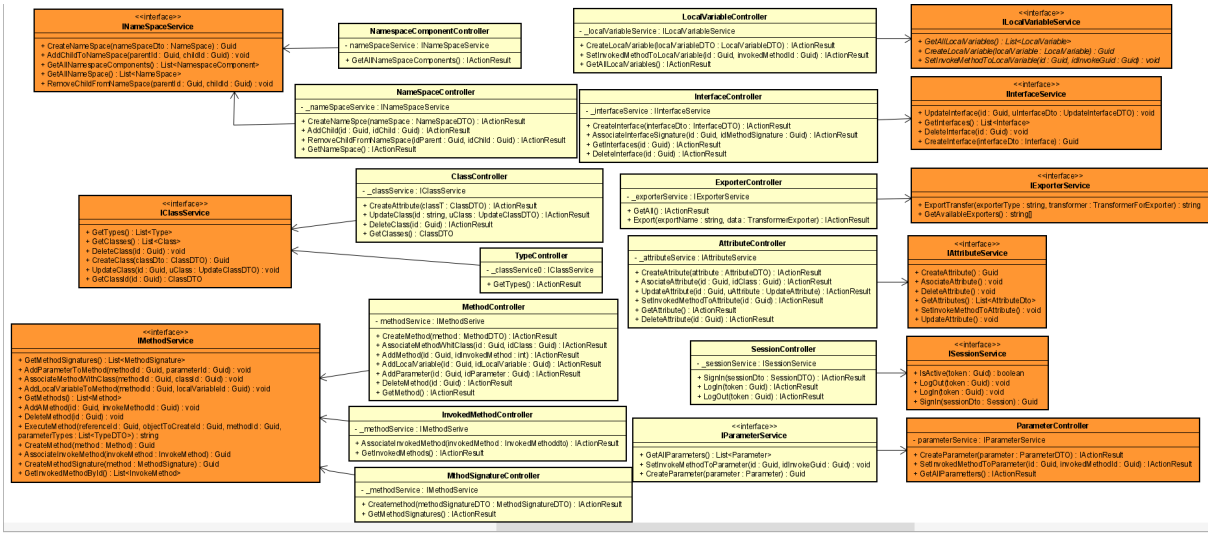
Simulator.ExportService: Se encarga de buscar dinámicamente exportadores disponibles en archivos DLL, instanciarlos y utilizarlos para exportar datos transformados según el tipo solicitado. Además, permite consultar los formatos de exportación soportados. Utiliza reflexión para cargar los exportadores y delega la ejecución de métodos a un servicio de métodos.

Web Api



Relación entre los controladores y las interfaces de los servicios que implementan

Las interfaces (representadas en naranja) se encuentra en un paquete separado llamado IBusinessLogic



Los controladores manejan las solicitudes HTTP recibidas y actúan como un punto de entrada a la aplicación. Interactúan con los servicios de la capa de lógica de negocio a través de las interfaces definidas en la misma capa, aplicando los filtros y validaciones necesarios antes de procesar las solicitudes. De esta manera, mantienen una separación clara entre la lógica de presentación y la lógica de negocio.

Filtros necesarios representados en color celeste:

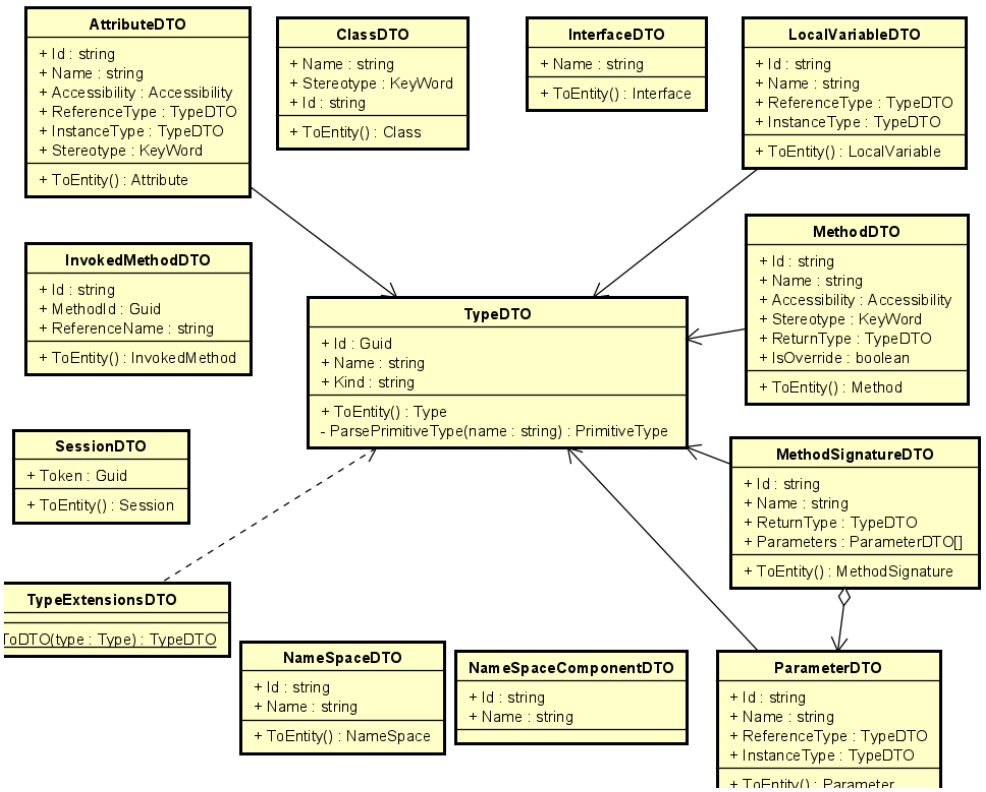
El ExceptionFilter se encarga de interceptar y manejar las excepciones que ocurren durante la ejecución de las solicitudes HTTP. Utiliza una estructura de control (como un switch) para identificar el tipo de excepción y devolver una respuesta adecuada al cliente. En caso de no reconocer el tipo de error, retorna un código de estado 500 (Internal Server Error) como respuesta genérica. Este enfoque centraliza el manejo de errores y mejora la claridad y robustez del sistema.

Además, se agruparon las excepciones por paquete especificamente en dominio y businesslogic dado que consideramos que los demas no lo necesitan esto quiere decir que cada módulo del sistema define sus propias excepciones específicas. Esta estrategia ofrece varias ventajas:

- Encapsulamiento de lógica de negocio: Las excepciones están estrechamente ligadas al contexto en el que ocurren, lo que permite que cada módulo exprese sus errores de forma precisa y clara.
- Reutilización y mantenibilidad: Agrupar excepciones facilita su mantenimiento y reutilización dentro del mismo paquete, reduciendo la duplicación y mejorando la organización del código.
- Evolución controlada: Si un módulo cambia, solo se deben ajustar las excepciones dentro de su propio paquete, sin afectar otras partes del sistema.
- Facilita las pruebas y la documentación: Tener las excepciones organizadas por paquete ayuda tanto en los tests unitarios como en la documentación del sistema, ya que es más fácil saber qué errores pueden surgir en cada contexto.

:El AuthorizationFilter es un filtro que permite restringir el acceso a ciertos endpoints de la API verificando que la petición incluya un token válido en el encabezado Authorization. Su principal función es comprobar que dicho token exista, esté bien formateado, sea un Guid válido y en este caso que esté dentro de una lista predefinida de sesiones permitidas. Si la validación falla, la ejecución de la acción se detiene y se devuelve un error 401 Unauthorized.

Diagrama DTO

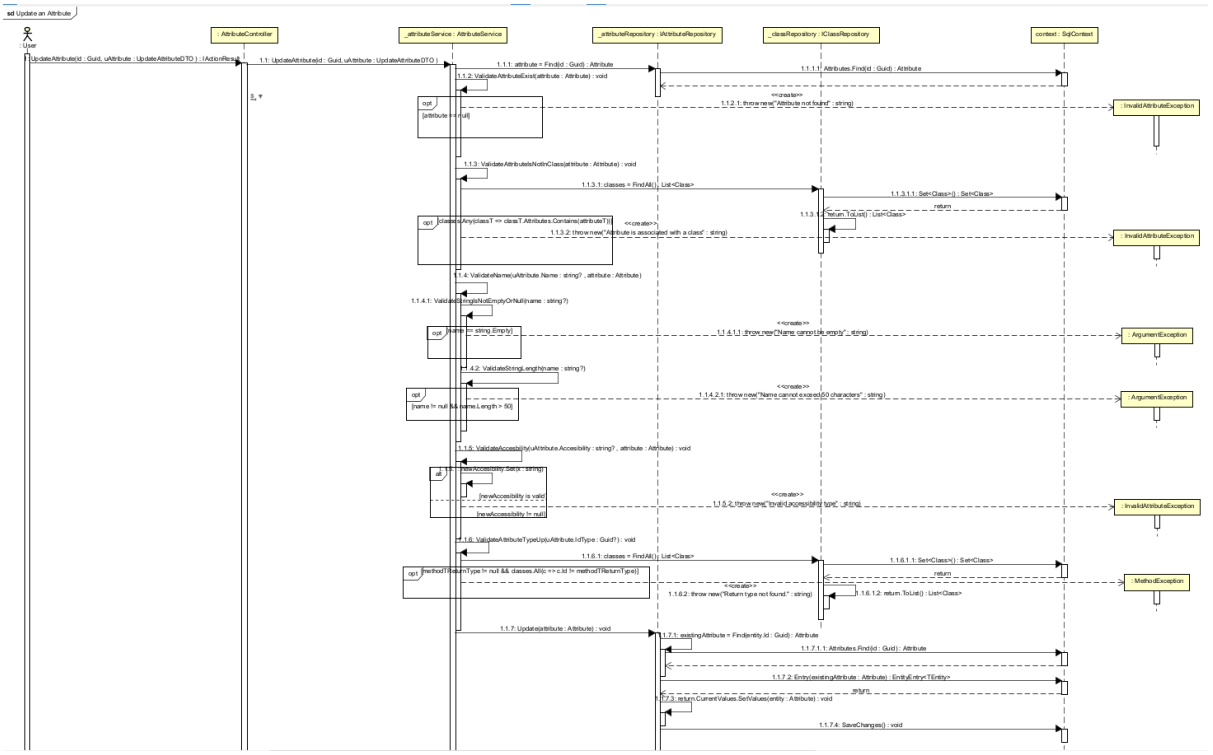


Este paquete va de la mano con el de WebApi, por ejemplo paara crear una clase el controlador recibe un ClassDTO en cambio el servicio que implementa recibe una Class ente intercambio se realiza con la funcion ToEntity() que se puede ver en cada dto, de esta forma se desacopla la capa de presentación de la capa de servicios.

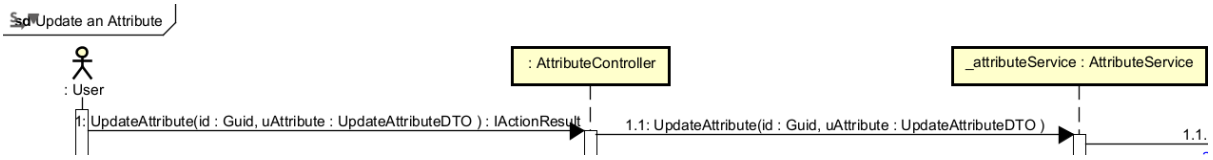
Esto favorece el principio de separación de responsabilidades, ya que cada capa se enfoca en lo suyo: el controlador maneja la comunicación con el cliente y el servicio la lógica de negocio.

Diagramas de Secuencia

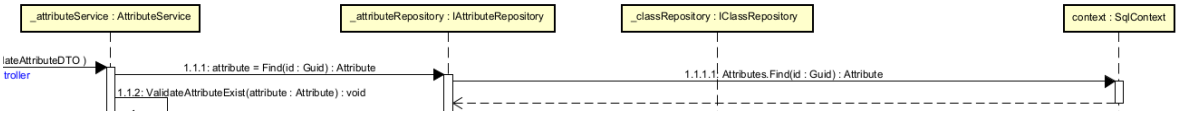
A continuación presentaremos un diagrama de secuencia para mostrar en detalle el proceso de la **actualización de un Atributo** (requerimiento UpdateAttribute):



Como se puede ver siempre los Controladores a través de la API serán los intermediarios entre los clientes (User) y nuestra lógica de negocio. En este caso específico con la llamada de “UpdateAttribute” el tiene como parámetro la id del atributo y un DTO que tendrá solamente la información para este proceso de manera que solo se comunique los datos que nos interesa de manera segura:

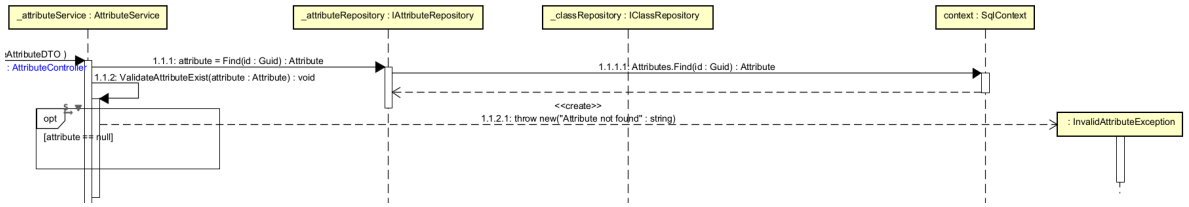


El **AttributeService** (encargado de la lógica de los procesos que involucran a los atributos) comienza buscando y guardando como una variable llamada “attribute” en el proceso **1.1.1** al atributo cuya id nos pasaron anteriormente y se encuentra guardada en nuestro SqlContext por nuestra base de datos:



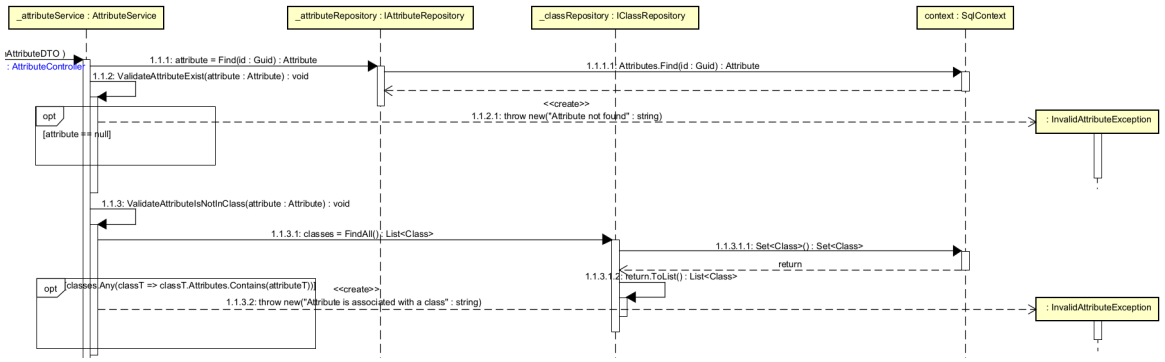
Una vez retornado y guardado el atributo a actualizar se realizarán varias verificaciones:

1) **ValidateAttributeExist:** El cual nos indicará con una excepción (InvalidAttributeExcepcion) para avisarle al cliente

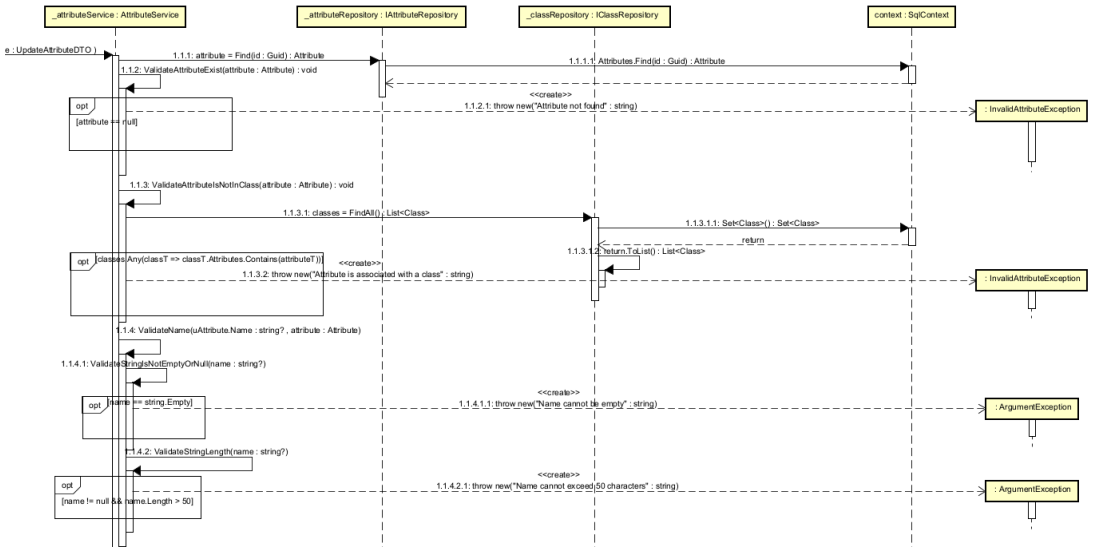


2) **ValidateAttributeIsNotInClass:** Primero guardará todas las clases almacenadas mediante el repositorio de clases y a continuación nos indicará con la misma excepción en el caso de que este atributo ya esté relacionado con una clase

---

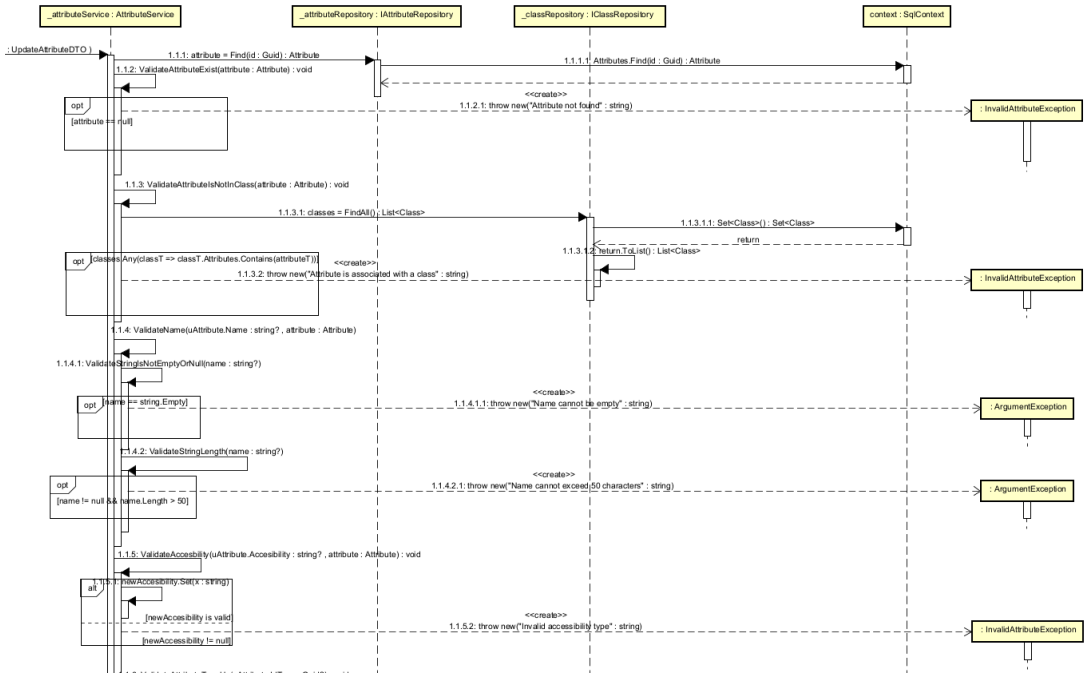


3) **ValidateStringIsNotEmptyOrNull:** Nos indicará usando excepciones si el nombre es nulo o si es demasiado largo

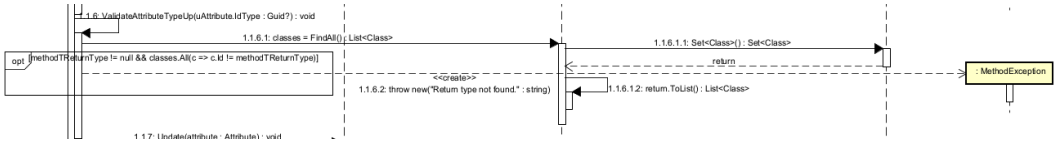


4) **ValidateAccesbility:** Verificará que la accesibilidad del atributo a actualizar (newAccesbility) es válido y se lo asignará de manera correcta (lo representé con el newAccesibiliy.Set) si es así, en caso contrario, se nos notificará con

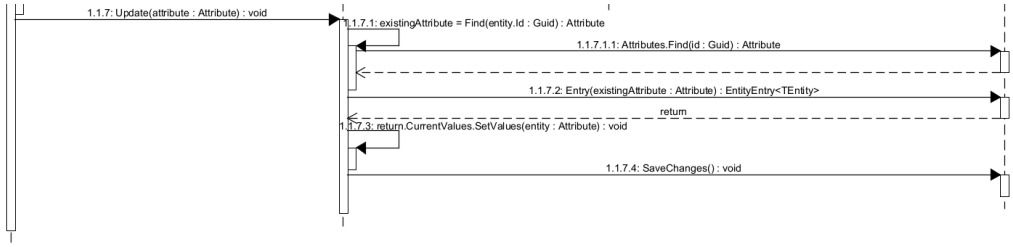
una excepción:



5) **ValidateAttributeTypeUp:** Finalmente tras pedirle todas clases al repositorio de clases se verificará si el tipo del atributo no es existente y en ese caso se nos devolverá una excepción



Ya habiendo realizado las verificaciones se buscará el atributo ya existente en la base de datos para a continuación actualizarlo con el atributo con los datos que nos indicó el cliente



En el caso que se cree bien se retornará como respuesta un:

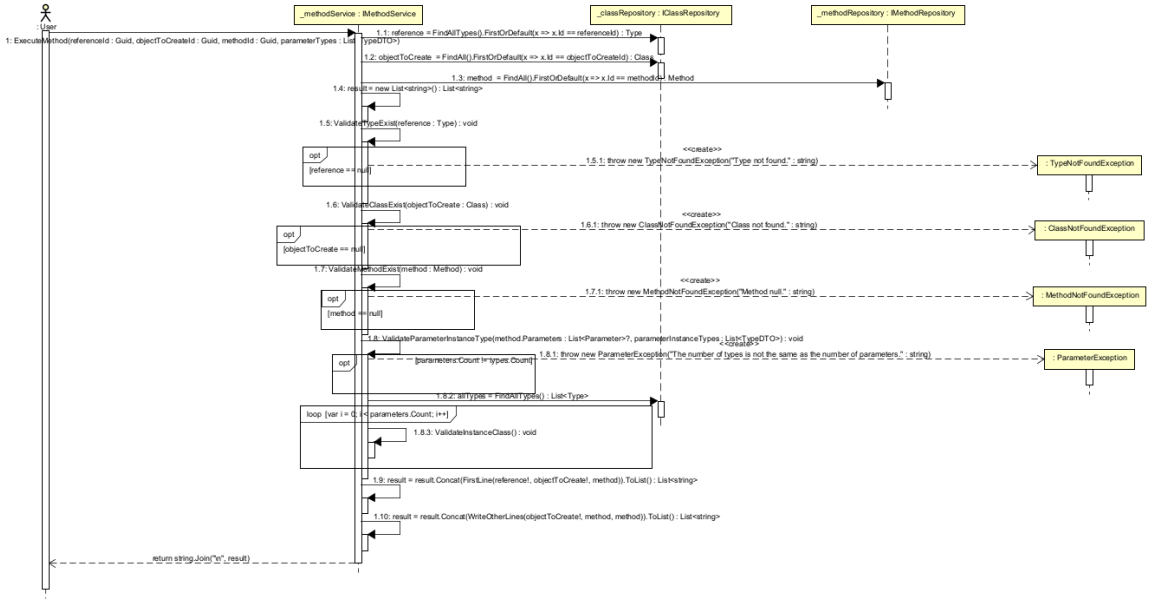
- StatusCode 200(Ok) y mensaje “Attribute updated succesfully”

Sino dependiendo de la excepción que se encuentre como mostré en el diagrama de secuencia el Filter se activará de la siguiente manera:

- InvalidAttributeException => StatusCode 400 con el mensaje de la excepción.
- MethodException => StatusCode 400 con el mensaje de la excepción.

.

Finalmente presentaremos un diagrama de secuencia resumido sobre la funcionalidad **ejecutar método**:

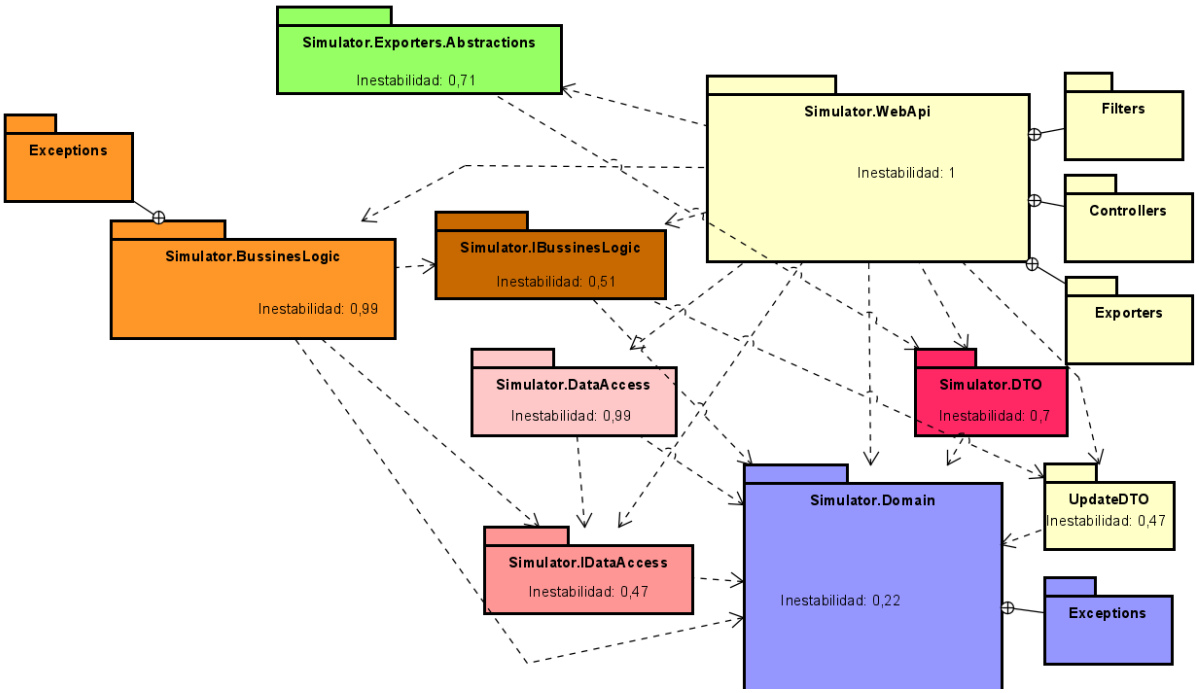


Vista de Desarrollo

Detalla la organización física del software en términos de sus componentes y cómo estos interactúan. Está orientada a los desarrolladores, enfocándose en cómo implementar el sistema.

Diagrama de paquetes

El diagrama de componentes muestra cómo se organiza el sistema en módulos (paquetes) y cómo interactúan entre sí.



Para mejorar la cohesión del sistema y aplicar correctamente el principio de Responsabilidad Única (SRP), se definió una arquitectura basada en servicios especializados. Cada paquete tiene a su cargo una responsabilidad clara y bien delimitada: la capa WebApi gestiona la interacción externa a través de controladores y filtros; BusinessLogic encapsula la lógica del negocio; DataAccess se encarga del acceso a los datos; Domain define las entidades centrales del modelo de negocio, y DTO actúa como capa de transporte de datos entre las distintas capas. Esta organización mejora la claridad del sistema, facilita su mantenimiento y simplifica las pruebas unitarias.

Además, se aplicaron los principios de Segregación de Interfaces (ISP) e Inversión de Dependencias (DIP) para reforzar la flexibilidad y escalabilidad del diseño. En el caso del ISP, se crearon interfaces específicas para cada servicio, como IBussinesLogic o IDataAccess, que exponen únicamente los métodos necesarios para cada contexto. Esto evita que los consumidores de esas interfaces deban depender de métodos que no utilizan, reduciendo así la complejidad de las dependencias y aumentando la claridad de la arquitectura.

En cuanto al principio de Inversión de Dependencias (DIP), se procuró que los módulos de alto nivel, como la lógica de negocio, no dependan directamente de módulos de bajo nivel, como el acceso a datos.Por ejemplo, el paquete BusinessLogic no depende directamente de DataAccess, sino que se apoya en la interfaz IDataAccess, definida en un módulo intermedio. Esto permite que la implementación concreta de acceso a datos sea inyectada externamente, promoviendo un bajo acoplamiento. Esta práctica no solo favorece la reutilización y el cambio de implementación con facilidad.

Además, se implementaron varios principios **GRASP** para reforzar la asignación adecuada de responsabilidades.

En cuanto al principio **Alta cohesión**, aunque la división en capas especializadas aporta cierta organización y una mayor facilidad a la hora de codear, no se logra una cohesión fuerte dentro de cada módulo como se puede ver en las métricas. tambien se procuró mantener un **bajo acoplamiento** entre capas mediante el uso de interfaces. Sin embargo, la capa WebApi presenta cierto acoplamiento con el resto de capas

DTOs

DTOs (Data Transfer Objects): Los DTOs son objetos que se utilizan para transferir datos entre las capas del sistema. Son especialmente útiles cuando es necesario desacoplar las entidades del dominio de los datos que se exponen o consumen en la Web API. Los DTOs permiten estructurar los datos de manera eficiente para la comunicación con los clientes y pueden ser personalizados según las necesidades de cada caso.

WebApi

Los controladores manejan las solicitudes HTTP recibidas y actúan como un punto de entrada a la aplicación. Interactúan con los servicios de la capa de lógica de negocio a través de las interfaces definidas en la misma capa, aplicando los filtros y validaciones necesarios antes de procesar las solicitudes. De esta manera, mantienen una separación clara entre la lógica de presentación y la lógica de negocio.

En este paquete también se encuentra el archivo Program.cs, este configura y pone en marcha la aplicación web, y utiliza inyección de dependencias como uno de los mecanismos clave para mantener un diseño limpio y flexible. A través del método builder.Services.AddScoped<Interface, Implementación>(), se registran las interfaces y sus clases concretas, permitiendo que los controladores y servicios dependan de abstracciones en lugar de implementaciones directas. Esto sigue el Principio de Inversión de Dependencias (DIP), facilitando la prueba unitaria, la extensión del sistema y el reemplazo de componentes sin afectar otras partes del código. Por otro lado pensamos modificar esto en un futuro hacer un paquete Factory que se encargue de estas implementación.

Métricas:

Para la obtención de estos resultados se utilizó la aplicación de NDepend.

Projects Metrics:

Projects	Afferent Coupling	Efferent Coupling	Relational Cohesion	Instability	Abstractness	Distance
Simulator.Domain v1.0.0.0	61	17	2	0.22	0.1	0.49
Simulator.IDataAcces v1.0.0.0	19	17	0.1	0.47	1	0.33
Simulator.DataAcces v1.0.0.0	1	137	3.56	0.99	0	0.01
Simulator.UpdateDTO v1.0.0.0	9	8	0.33	0.47	0	0.37
Simulator.DTO v1.0.0.0	16	37	0.62	0.7	0	0.21
Simulator.Exporters.Abstractions v1.0.0.0	4	10	0.5	0.71	0.5	0.15
Simulator.IBussinesLogic v1.0.0.0	23	24	0.11	0.51	1	0.36
Simulator.BussinesLogic v1.0.0.0	1	94	1.03	0.99	0	0.01
Simulator.WebApi v1.0.0.0	0	161	1.04	1	0	0
Simulator.HTMLExporter v1.0.0.0	0	6	1	1	0	0

Principios de Paquetes y Análisis de las métricas

La **Relational Cohesión (H)** mide qué tan relacionadas están entre sí las clases de un mismo paquete, y en términos generales se considera saludable que los valores estén entre 1,5 y 4. En nuestro proyecto, la mayoría de paquetes no alcanzan ese rango, presentando valores menores a 1,5. Esto indica que sus clases no interactúan frecuentemente entre sí, lo cual podría sugerir baja cohesión interna.

El Principio de Clausura Común (CCP), que establece que las clases dentro de un módulo deben cambiar por las mismas razones, implica que un paquete debe agrupar clases que estén cohesionadas en términos de motivos de cambio. Para cumplir con este principio, el valor de H (medida de cohesión del paquete) debería estar dentro de un rango adecuado.

Dado que decidimos plantear nuestro proyecto con una arquitectura por capas (por ejemplo, separando Domain, DTOs, BusinessLogic, WebAPI, etc.), esta decisión no necesariamente favorece el CCP, ya que agrupar por tipo de responsabilidad (en lugar de motivo de cambio) puede provocar una menor cohesión interna. Sin embargo, esto no significa que esté mal, sino que fue una elección deliberada para lograr una mejor organización del código, facilitar el desarrollo, las pruebas, y mantener una estructura clara y predecible.

Aunque en muchos paquetes esta estructura por capas puede dificultar el cumplimiento del CCP, en el caso del paquete Domain, esto no sucede. De hecho, su valor de H es 2, lo que indica que sus clases están bien cohesionadas y cambian por motivos similares. Esto es esperable, ya que las clases del dominio suelen estar fuertemente relacionadas entre sí y

modelan conceptos estables del negocio. Por lo tanto, Domain es un buen ejemplo de cómo un paquete puede cumplir con el CCP incluso dentro de una arquitectura por capas.

La estructura por capas cumple con el **Principio de Reutilización Común (Common Reuse Principle, CRP)** porque agrupa clases y componentes que tienden a cambiar y reutilizarse juntos dentro de la misma capa. Por ejemplo, la capa de lógica de negocio contiene solo la lógica central del dominio, mientras que la capa de acceso a datos se encarga exclusivamente de la persistencia. Así, cuando una funcionalidad necesita ser reutilizada (por ejemplo, una validación o una operación de negocio), se reutiliza toda la capa correspondiente, evitando dependencias innecesarias con otras capas.

Sin embargo, es importante tener en cuenta que el cumplimiento del CRP y del CCP también depende en gran medida del tipo de cambio que se espera en el sistema

Inestabilidad

Conceptos necesarios:

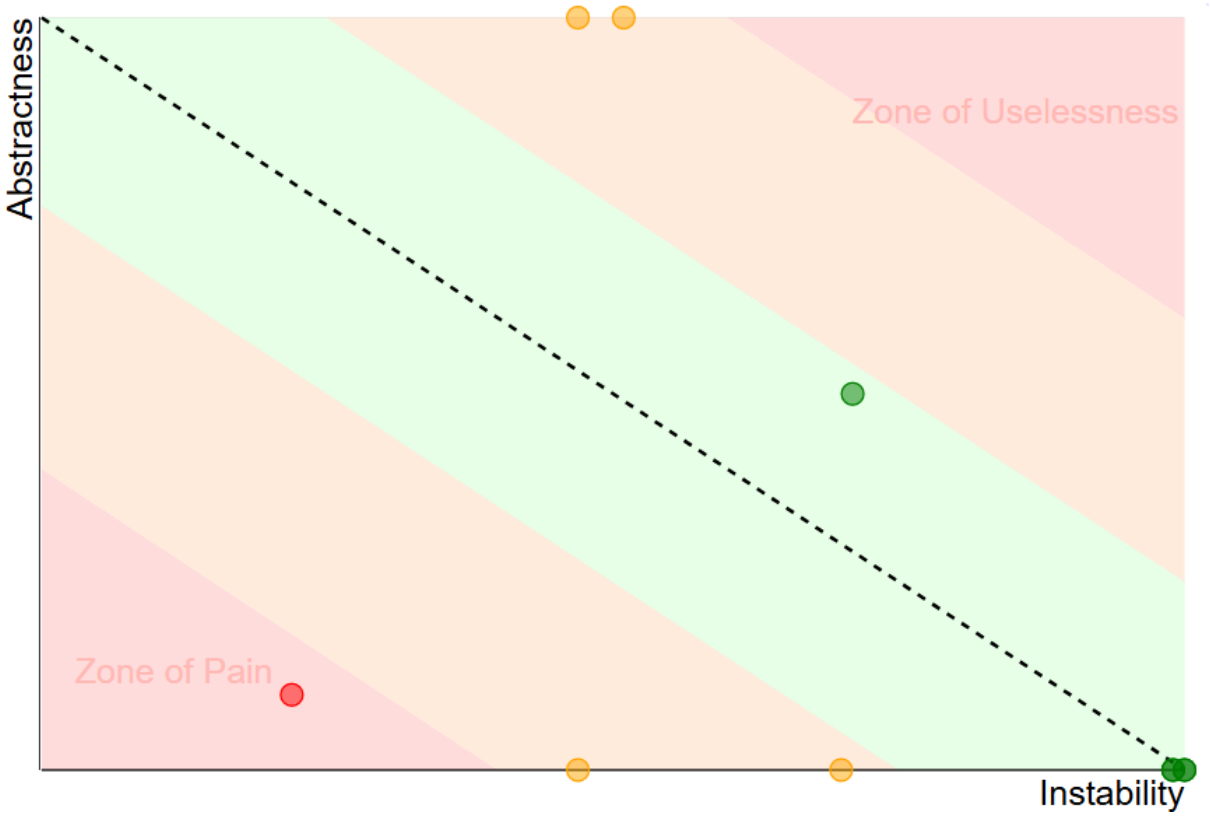
El **acoplamiento aferente** (Ca) mide cuántos módulos dependen del módulo actual. Si muchos lo utilizan, significa que es un módulo importante y estable, ya que cualquier cambio en él puede afectar a otros. Por otro lado, el **acoplamiento eferente** (Ce) indica cuántos módulos externos utiliza el módulo actual. Si depende de muchos, es más propenso a romperse si alguno de esos cambia, lo que lo hace más inestable.

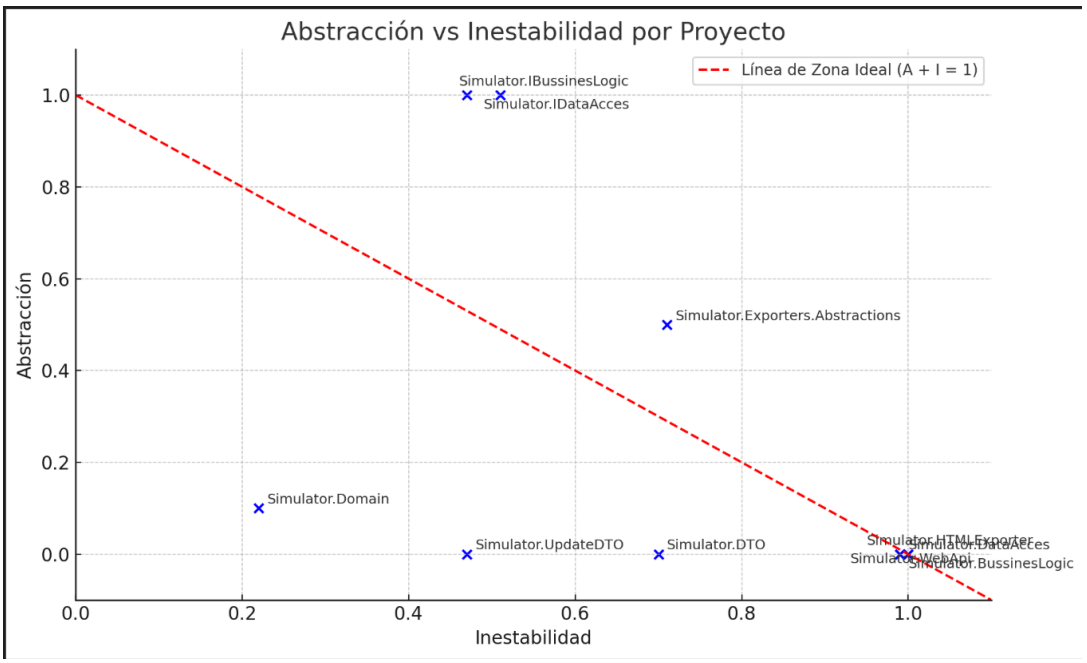
Esta métrica indica cuán inestable es un paquete y se calcula $I = \frac{Ce}{Ca+Ce}$. Más cercano a 0 indica mayor estabilidad.Luego de analizar los valores, podemos afirmar que se cumple el **Principio de Dependencias Estables (SDP)**, ya que todos los paquetes de nuestra aplicación dependen de paquetes más estables que él mismo, como se puede ver en el diagrama de paquetes, en la página en el que también representamos la inestabilidad de cada uno para que se vea mas claro.

Abstracción

La abstracción indica cuán abstracto es un paquete. Un paquete debe ser tan abstracto como estable. Cuando este valor está más cerca de 1, entonces el paquete es más abstracto.
Analizando los valores, notamos que nuestros paquetes de interfaces son los más abstractos (cercanos a 1) y los que implementan estas interfaces o tienen implementaciones concretas (como BusinessLogic) son cercanos a 0.Esto es lógico ya que el sistema fue pensado para funcionar de esta manera.

Abstraction vs Inestability





Principio de Abstracciones Estables:

Este principio analiza si los módulos más estables también son los más abstractos. La métrica Distance mide qué tan alejados están los paquetes del equilibrio ideal entre abstracción (A) e inestabilidad (I).

En nuestro análisis, observamos que el paquete Domain se encuentra en la zona de dolor (Zone of Pain), lo que significa que es altamente estable pero poco abstracto. Esto implica

que, aunque muchos otros módulos dependen de él, resulta difícil de extender y cualquier cambio en él podría impactar considerablemente en el resto del sistema.

Por otro lado, es positivo destacar que no tenemos paquetes ubicados en la zona de poca utilidad , lo que sugiere que todos los paquetes del sistema son relevantes y aportan funcionalidad concreta a la aplicación. Aunque algunos como IDataAcces se encuentran cerca y hay que tenerlo en cuenta.

Finalmente, los ubicados en la zona verde presentan un buen equilibrio entre estabilidad y abstracción. Su distancia al ideal es baja, lo que indica que cumplen adecuadamente con el principio de abstracciones estables.

Otros principios:

Principio de equivalencia de reuso/liberación (REP)

Esto indica que las clases de un paquete deben reutilizarse y liberarse juntas. En nuestro proyecto, este principio no se cumple completamente.

- En WebApi, se agrupan Controllers y Filters, que no siempre cambian ni se usan en conjunto.
- En Domain y BusinessLogic, se incluyen paquetes de excepciones que se usan de forma transversal y no están directamente relacionadas con el resto del contenido.

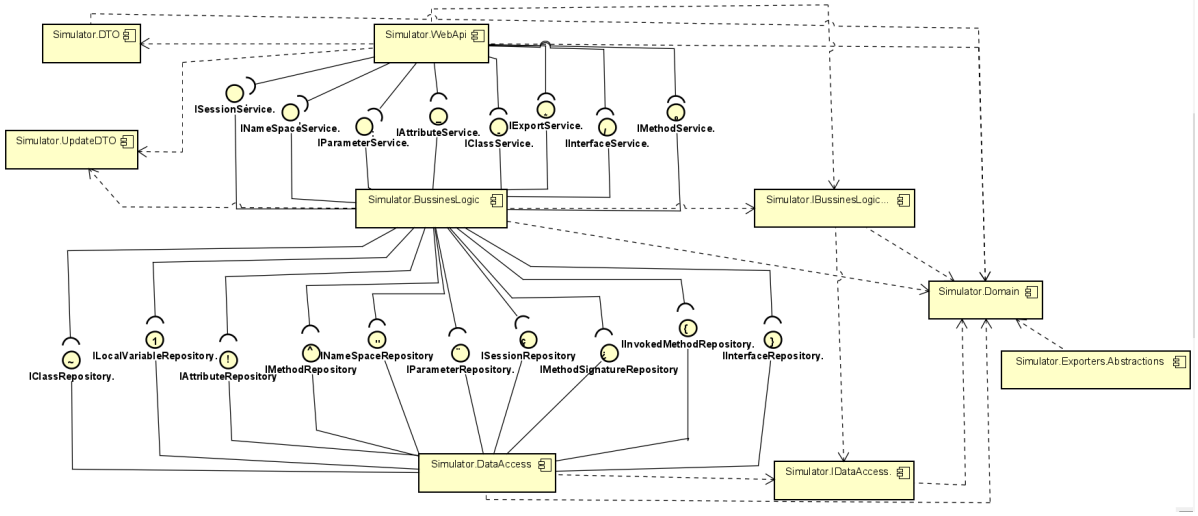
Sin embargo, no siempre es indispensable cumplir estrictamente con REP no consideramos óptimo separar controllers y filters del paquete de web api y en caso de las excepciones priorizamos la separación de preocupaciones entre capas dándole mayor importancia al principio GRASP, Bajo Acoplamiento ya que tener un único paquete de Excepciones generaría que este muy acoplado.

A nivel de acoplamiento:

Principio de Dependencia A-cíclica (ADP)

Este proyecto cumple con el Principio de Dependencia A-cíclica (ADP) que establece que no debe haber ciclos en las dependencias entre paquetes. Esto contribuye a un diseño más estable y mantenible, permitiendo que los paquetes evolucionen de forma independiente, reduciendo la complejidad del sistema. Esto se puede demostrar debido a que el proyecto fue implementado en .NET 8.0 y este no permite las dependencias cíclicas.

Diagrama de componentes



El siguiente diagrama de componentes representa la arquitectura general del sistema *Simulator*, estructurado en capas bien definidas que siguen principios de separación de responsabilidades, modularidad y reutilización.

Este modelo muestra los principales componentes del sistema, así como las relaciones y dependencias entre ellos, facilitando el entendimiento de la interacción entre los distintos módulos en tiempo de ejecución.

El diagrama utiliza símbolos estandarizados para representar la relación entre los módulos mediante interfaces.

Interfaces provistas: Representadas por un círculo completo (tipo lollipop), indican que el componente expone dicha interfaz y proporciona su implementación. Por ejemplo, el componente Simulator.BusinessLogic provee la interfaz IClassService, la cual puede ser utilizada por otros componentes como Simulator.WebApi.

Interfaces requeridas: Representadas por un semicírculo, indican que el componente depende de una interfaz cuya implementación es provista por otro. Por ejemplo, Simulator.BusinessLogic requiere la interfaz IClassRepository, la cual es implementada por Simulator.DataAccess.

Este enfoque promueve un diseño desacoplado, facilitando la reutilización de componentes, la independencia entre capas.

Modelo de Tablas



Mejoras a nivel de diseño

Cambio en las entidades Attribute, LocalVariable y Parameter
Estas entidades ahora contienen una lista de InvokedMethod en lugar de estar limitadas sólo una. Esto permite modelar de forma más precisa el simulador.

Refactorización de servicios (ParameterService, LocalVariableService)
Se reorganizó la lógica de negocio distribuyendo responsabilidades de forma más clara: se crearon servicios dedicados para parámetros y variables locales, reduciendo la carga del

MethodService. Esto mejora la separación de responsabilidades y hace que cada servicio sea más coherente, testeable y mantenible. esta decisión aplica el principio GRASP de Alta Cohesión, ya que cada clase tiene responsabilidades bien definidas y relacionadas, lo que facilita su comprensión, reutilización y evolución en el sistema.

El paquete de Simulator.BusinessLogic no depende del paquete Simulattor.DTOs

Esto implica una reducción significativa del acoplamiento entre capas, lo que favorece una arquitectura más limpia y flexible. Al eliminar esta dependencia, la lógica de negocio se vuelve independiente de cómo se representan o transfieren los datos hacia el exterior del sistema. Esta mejora promueve una clara separación de responsabilidades y fortalece el principio de inversión de dependencias, alineándose con buenas prácticas de diseño como la arquitectura por capas.

La clase Type abstracta

Esto representa una decisión de diseño coherente con el hecho de que Type es un concepto general que no debería ser instanciado directamente. Al ser abstracta, obliga a que siempre se utilicen subclases concretas, esto habilita el uso de polimorfismo: distintos comportamientos pueden ser definidos en las subclases sin que el resto del sistema necesite conocer sus diferencias. De esta manera, se mejora la extensibilidad del sistema, ya que se pueden incorporar nuevos tipos concretos sin modificar la clase base, respetando así el principio abierto/cerrado (OCP) del diseño orientado a objetos.

Estas decisiones de diseño y el uso consciente de principios y patrones permiten construir un sistema más robusto, extensible y fácil de mantener.

Anexo:

Coverage del ultimo PR

Assembly	Line Coverage	Branch Coverage	Health (over 90.0%)
Simulator.BussinesLogic	95.81%	90.07%	✅
Simulator.DataAcces	100.00%	93.75%	✅
Simulator.Domain	99.00%	92.31%	✅
Simulator.DTO	98.89%	84.21%	❌
Simulator.Exporters.Abstractions	100.00%	100.00%	✅
Simulator.UpdateDTO	100.00%	100.00%	✅
Simulator.WebApi	99.59%	91.43%	✅
----------	----------	----------	----------
Summary	**99.04%**	**93.11%**	✅

Minimum allowed code coverage is 90.0%

En caso de los DTOs aunque se haya realizado TDD no logramos deducir que casos nos faltaron al final para mejorar este campo y por cuestión de tiempo decidimos darle prioridad a otros campos dado que un porcentaje de 84.21 no está alejado de lo que se buscaba.

Especificación de la api con los endpoint agregados para esta entrega

Recurso	Verbo	URI	Header	Body	Respuesta	Status Code	Descripción
Class	GET	api/v2/classes		---	200 - [{id : string, name : string, stereotype : KeyWord}]		Trae todas las clases definidas en el sistema, incluyendo su nombre y estereotipo.
Attribute	GET	api/v2/attributes			200 - [{id : string, name : string, Accessibility : Accesibility, ReferenceType : Type, InstanceType : Type, stereotype : Stereotype}]		Devuelve todos los atributos de clases, con su accesibilidad, tipos de referencia e instancia, y estereotipo.
Export	GET	api/v2/exporters			200		Trae todos los ensamblados en tiempo de ejecucion
Export	POST	api/v2/exporters			201	401- "API Key inválida o ausente". 400-"Exporter type exporterType not found" 404- "Type not found." 404-"Method null."404-"Class null." 400-"The number of types is not the same as the number of parameters" 400-"Instance Type is not a subclass of Reference Type" 400-"Invalid reference type or instance type. If one is primitive, both should be primitive. If both are, then they must be the same."	

Interface	POST	api/v2/interfaces	{ name : string}	201 { iinterfaceId : Guid }	400 - "The interface name cannot be empty" 400- "The interface name must start whit an uppercase letter. 400 - "There interface name must not contain spaces." 400 - "There is already a type with this name"	Crea interfaces
Interface	POST	api/v2/interfaces/{id}/methodSignatures/{idMethodSignature}		201	404 - "Method signature not found" 404- "Interface not found" 409-"Interface already have this mehodsignature"	Le asocia methodssignatures a una interface
Interface	DELETE	api/v2/interfaces/{id}		204	409 - "Interface is associated whit a class" 404- "There is no interface with this id. "	Borra una interface
Interface	GET	api/v2/interfaces		200 - [{id : string, name : string, Kind : string}]		Lista todas las interfaces disponibles, incluyendo su tipo o naturaleza (`Kind`).
InvokedMethods	GET	api/v2/invokedMethods		200 - [{id : string, MethodId : strin, ReferenceName : string}]		Devuelve todos los métodos invocados desde otros métodos, con su identificador, el ID del método destino y el nombre con el que fue invocado.
Local Variable	GET	api/v2/localVaraibles/		200 - [{id: string, name: string, ReferenceType: {name: string, kind: string},InstanceType: {name: string, kind: string}}]		Obtiene todas las variables locales declaradas dentro de métodos, junto a sus tipos de referencia e instancia.
Method	GET	api/v2/methods		200 - [{id: string, name: string, Accessibility: Accessibility, ReturnType: {name: string, kind: string}, stereotype: Stereotype}]		Recupera todos los métodos definidos, incluyendo su nombre, visibilidad, tipo de retorno y estereotipo.
MethodSignature	POST	api/v2/methodSignatures	{ id : string, name : string, ReturnType : TypeDTO, IntanceType : TypeDTO }	201 { methodSignatureId : Guid }	400 - "MethodSignature name cannot be null." 404- "MethodSignature cannot be null."	Crea un method signature
MethodSignature	GET	api/v2/methodSignatures		200 - [{id: string, name: string, ReturnType: {id: Guid, name: string, kind: string}}]		Lista todas las firmas de métodos que pueden ser implementadas por interfaces.
NamSpaceComponent	GET	api/v2/nameSpacesComponents		200 - [{id: string, name: string}]		Obtiene todos los componentes que están organizados dentro de espacios de nombres.
NameSpace	POST	api/v2/nameSpaces	{ id : string, name : string}	201 { nameSpaceId : Guid }	404- "NameSpace already exist"	Crea name space
NameSpace	POST	api/v2/nameSpaces/{parentID}/addChilds/{childId}		201	404- "NameSpace does not exist" 400-"A namespace cannot have the name Object" 404-"NameSpace does not exist" 400 - "Cannot relate a NameSpace with itself " 400-"Circular reference detected" 400-"This namesace already contains a child whit the same name."	Le asocia un namespaceocmponent como hijo a un componente que actura como su padre
NameSpace	DELETE	api/v2/nameSpaces/{parentID}/removeChilds/{childId}		204	404- "NameSpace does not exist" 404-"NameSpace does not exist" 400 - "Cannot relate a NameSpace with itself " 400-"Child is not part of the specified parent."	Se elimina como hijo un namespaceocmponent
NameSpace	GET	api/v2/nameSpaces		200 - [{id: string, name: string}]		Devuelve todos los espacios de nombres definidos,
Parameter	GET	api/v2/parameters		200 - [{id: string, name: string, ReferenceType: {name: string, kind: string}, InstanceType: {name: string, kind: string}}]		Trae todos los parámetros de métodos, con su nombre y los tipos de referencia e instancia correspondientes.
Session	POST	api/v2/sessions	{ Token : Guid}	201 { token : Guid }	404- "A session wtih this token already exists."	Crea una session inactiva
Session	PATCH	api/v2/sessions/logins/{token}		200	400- "Session already active."	Activa una session
Session	PATCH	api/v2/sessions/logouts/{token}		200	400- "Session already active."	Desactiva una session
Type	GET	api/v2/type		200 - [{name: string, kind: string}]		Devuelve todos los tipos definidos (primitivos, class, interface)

Métricas de forma más específica

Type Name	Afferent Coupling	Efferent Coupling	Type Namespace
Type	36	11	Simulator.Domain
NamespaceComponent	27	10	
InvokedMethod	19	8	Simulator.Domain
Parameter	15	15	Simulator.Domain
SqlContext	47	71	Simulator.DataAcces
Method	12	20	Simulator.Domain
TypeDTO	17	21	
NameSpace	8	11	Simulator.Domain
Interface	14	11	Simulator.Domain
LocalVariable	12	15	Simulator.Domain
Attribute	12	20	Simulator.Domain
Session	7	7	Simulator.Domain
Class	14	18	Simulator.Domain
KeyWord	14	2	Simulator.Domain
MethodSignature	11	12	Simulator.Domain
PrimitiveType	8	2	Simulator.Domain
Accessibility	10	2	Simulator.Domain
UpdateInterfaceDTO	3	5	Simulator.UpdateDTO
IClassRepository	9	7	Simulator.IDataAcces
UpdateAttributeDTO	3	7	Simulator.UpdateDTO
ITransformerExporter	2	3	Simulator.Exporters.Abstractions
IInvokedMethodRepository	6	5	Simulator.IDataAcces
LocalVariableException	3	4	Simulator.Domain.Exception
IMethodService	6	9	Simulator.IBussinesLogic
ParameterException	2	4	Simulator.Domain.Exception
ISessionService	3	5	Simulator.IBussinesLogic
ISessionRepository	3	5	Simulator.IDataAcces
INameSpaceService	4	6	Simulator.IBussinesLogic
IInterfaceRepository	5	6	Simulator.IDataAcces
IClassService	4	7	Simulator.IBussinesLogic
MethodException	2	4	Simulator.Domain.Exception

AttributeException	2	4	Simulator.Domain.Exception
InvokedMethodNotFoundException	4	4	Simulator.BussinesLogic.Exception
ClassException	1	4	Simulator.Domain.Exception
IExportService	3	4	Simulator.IBussinesLogic
IAttributeRepository	4	6	Simulator.IDataAcces
IParameterRepository	4	5	Simulator.IDataAcces
ParameterDTO	2	12	Simulator.DTO
ILocalVariableRepository	4	5	Simulator.IDataAcces
IMethodSignatureRepository	4	5	Simulator.IDataAcces
INameSpaceRepository	3	6	Simulator.IDataAcces
IParameterService	3	5	Simulator.IBussinesLogic
IMethodRepository	4	6	Simulator.IDataAcces
ILocalVariableService	3	5	Simulator.IBussinesLogic
IInterfaceService	3	6	Simulator.IBussinesLogic
TypeNotFoundException	3	4	Simulator.BussinesLogic.Exception
IAttributeService	3	6	Simulator.IBussinesLogic
ClassNotFoundException	3	4	Simulator.BussinesLogic.Exception
ExceptionFilter	1	14	Simulator.WebApi.Filters
InvalidAttributeException	2	4	Simulator.BussinesLogic.Exception
SessionDTO	1	7	Simulator.DTO